Project Report
on

Food Ordering
&
Delivery System

Submitted By

Arnab Lala (25MCI10232)

Subject Name: Technical Training – I
Subject Code: CAP-652

in partial fulfilment for the award of the degree of
MCA
IN
Artificial Intelligence and Machine Learning
University Institute of Computing
Department of Carrier Planning and Development

Academic Year: 2025–26

Under the Guidance of:
Mr. Shalabh Bhatiya
(E18471)

ACKNOWLEDGEMENT

We would like to express our sincere gratitude and appreciation to all those who have helped and guided us throughout the completion of this project on **Food Ordering System**.

First and foremost, we are deeply grateful to our faculty mentor for their constant encouragement, invaluable feedback, and academic guidance. Their expertise in database management systems and relational database design helped us navigate through the complexities of this project with confidence.

We would also like to acknowledge our fellow classmates whose constructive suggestions and collaborative spirit enriched our understanding of database normalization, trigger-based automation, and SQL query optimization.

Special thanks to our families for their unwavering moral support and encouragement throughout the duration of this academic project.

This project has been an invaluable learning experience, enhancing our practical understanding of PostgreSQL, schema design, stored procedures, views, and complex query formulation in a real-world context.

UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+

CERTIFICATE

This is to certify that the project entitled **"FOOD ORDERING SYSTEM USING ADBMS (POSTGRESQL)"**
has been successfully completed by the following students as part of their academic curriculum for the academic session 2025–2026 for Second Semester. The work submitted is original and has not been submitted elsewhere for any degree or certification.

Students:

Arnab Lala	MCA – AI&ML	25MCI10232

Faculty Guide

Signature: _____

Date: _____

ABSTRACT

The Food Ordering System is a relational database-driven application designed to efficiently manage and streamline the process of online food ordering and delivery. Traditional food ordering methods often involve manual coordination, inefficient tracking, and lack of centralized data management, leading to delays, errors, and poor user experience.

This project presents a comprehensive PostgreSQL-based database schema that models key entities such as users, restaurants, menu items, orders, order items, payments, delivery tracking, and reviews, each represented through normalized relational tables. The system ensures referential integrity using foreign key constraints, automates calculations using database triggers, and enables flexible data retrieval through SQL views and stored functions.

Key features of the system include automatic subtotal calculation and order total updates using triggers, a real-time view for restaurant ratings and summaries, a PL/pgSQL function to retrieve order history of users, and constraint enforcement to maintain valid data entries. The project also demonstrates a wide range of SQL queries including data retrieval, updates, deletions, aggregation, and multi-table joins.

The database supports multiple users, restaurants, and orders, tracks payment statuses and delivery progress, and allows users to provide ratings and reviews. The system proves effective in managing food ordering workflows, maintaining consistency, and generating useful analytical insights.

This project serves as a practical demonstration of core database management principles including entity-relationship modeling, normalization up to Third Normal Form (3NF), procedural SQL, trigger-based automation, and query optimization.

TABLE OF CONTENT

Sr. No.	Title	Page No.
1	Introduction	6
2	Objectives	7
3	System Requirements	7
4	Entity-Relationship Description	8
5	Table Structures & Creation	9
6	Constraints & Trigger & Function	10
7	Data Insertion (INSERT Queries)	11
8	SELECT Queries with Outputs	14
9	UPDATE Queries with Outputs	16
10	DELETE & Other Queries with Outputs	17
11	Conclusion	18
12	References	19

1. INTRODUCTION

The Food Ordering System is an essential application in today's digital world, where online food delivery services have become increasingly popular. Managing users, restaurant menus, orders, payments, and deliveries manually is inefficient and prone to errors. Therefore, a structured and automated database system is required to ensure accuracy, speed, and scalability.

This project is developed using PostgreSQL, a powerful open-source relational database management system. It models real-world food ordering scenarios where users browse restaurants, select menu items, place orders, make payments, and track delivery status.

The database schema consists of multiple interconnected tables such as users, restaurants, menu_items, orders, order_items, payments, delivery_tracking, and reviews. These tables are linked using foreign keys to maintain referential integrity and ensure consistency.

Advanced database features implemented in this project include:

- Triggers — for automatic subtotal and total amount calculations
- Stored Functions — to retrieve user order history
- Views — to generate restaurant rating summaries
- Check Constraints — to validate inputs such as rating and status values

The SQL implementation covers essential operations including SELECT queries with joins and aggregation, UPDATE and DELETE operations, and advanced queries using STRING_AGG for consolidated reporting.

2. OBJECTIVES AND SYSTEM REQUIREMENTS

2.1 Objectives

The primary objectives of this Food Ordering System database project are:

- To design a relational database schema for managing a food ordering system
- To implement automation using triggers for order calculations
- To maintain referential integrity between users, orders, and restaurants
- To manage menu items, payments, and delivery tracking efficiently
- To implement review and rating systems for restaurants
- To demonstrate SQL operations including SELECT, INSERT, UPDATE, DELETE
- To create stored functions for retrieving user-specific data
- To optimize queries using indexing.

2.2 System Requirements

Component	Specification
Database	PostgreSQL 14+ / pgAdmin 4
Language	SQL, PL/pgSQL
OS	Windows 10/11, Ubuntu 20.04+
RAM	Minimum 4 GB
Storage	Minimum 500 MB free space
Tools	pgAdmin 4, VS Code (SQL extension), Terminal
Network	Local (standalone) or LAN

3. ENTITY-RELATIONSHIP DESCRIPTION

The Hostel Management System is built on four primary entities, each mapped to a relational table. The relationships between these entities are defined using primary and foreign keys, ensuring referential integrity and consistent data flow.

3.1 Entities and Their Attributes

Entity	Primary Key	Key Attributes	Relationship
Users	user_id	name, email, phone, address	Has many orders, many reviews
Restaurants	restaurant_id	name, location, rating	Has many menu items, many reviews
Menu_Items	item_id	restaurant_id, item_name, price	Belongs to one restaurant
Orders	order_id	user_id, order_date, status, total_amount	Belongs to one user, has many order_items
Order_Items	order_item_id	order_id, item_id, quantity, price, subtotal	Links Orders ↔ Menu_Items
Payments	payment_id	order_id, payment_method, payment_status, amount	Belongs to one order
Delivery_Tracking	delivery_id	order_id, delivery_status, delivery_time	One-to-one with orders
Reviews	review_id	user_id, restaurant_id, rating, review_text	Links Users ↔ Restaurants

3.2 Relationships Summary

Users to Orders: One user can place multiple orders (one-to-many). Restaurants to Menu_Items: One restaurant can have multiple menu items (one-to-many). Orders to Order_Items: One order can contain multiple food items (one-to-many). Users to Reviews: One user can give multiple reviews (one-to-many). Restaurants to Reviews: One restaurant can receive multiple reviews (one-to-many).

The **order_items table acts as a junction entity** that resolves the many-to-many relationship between orders and menu items.

3.3 Constraints Applied

Table	Constraint Type	Description
users	PRIMARY KEY	user_id uniquely identifies each user
restaurants	PRIMARY KEY	restaurant_id uniquely identifies each restaurant
menu_items	FOREIGN KEY	restaurant_id references restaurants(restaurant_id)
orders	FOREIGN KEY	user_id references users(user_id)
order_items	FOREIGN KEY	order_id references orders(order_id)
order_items	FOREIGN KEY	item_id references menu_items(item_id)
payments	FOREIGN KEY	order_id references orders(order_id)
delivery_tracking	FOREIGN KEY	order_id references orders(order_id)
reviews	FOREIGN KEY	user_id references users(user_id)
reviews	FOREIGN KEY	restaurant_id references restaurants(restaurant_id)
menu_items	CHECK	price > 0
reviews	CHECK	rating BETWEEN 1 AND 5

4. TABLE STRUCTURES & CREATION

4.1 Users Table

The users table stores essential information about each customer including their name, email, phone number, and address. The user_id is an auto-incrementing primary key used to uniquely identify each user in the system**.**

```
CREATE TABLE users (
    user_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    phone VARCHAR(15) UNIQUE,
    address TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

4.2 restaurants Table

The restaurants table maintains a record of all restaurants, including their name, location, and rating. The rating field ensures valid values and helps in evaluating restaurant performance within the system.

```
CREATE TABLE restaurants (
  restaurant_id SERIAL PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  location VARCHAR(150),
  rating DECIMAL(2,1) CHECK (rating >= 0 AND rating <= 5),
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

4.3 menu_items Table

The orders table records each order placed by a user along with details such as order date, status, and total amount. It references the users table via a foreign key, ensuring that only valid user IDs can be used for creating orders.

```
CREATE TABLE menu_items (
  item_id SERIAL PRIMARY KEY,
```

```
    restaurant_id INT REFERENCES restaurants(restaurant_id) ON DELETE CASCADE,

    item_name VARCHAR(100) NOT NULL,

    price DECIMAL(6,2) CHECK (price > 0),

    is_available BOOLEAN DEFAULT TRUE

);
```

4.4 reviews Table

The complaints table captures all student-reported issues, the nature of the complaint, and the current resolution status. Status defaults to 'Pending' when a new complaint is filed and can be updated to 'In Progress' or 'Resolved' as the issue is handled.

```
CREATE TABLE complaints (
    complaint_id SERIAL PRIMARY KEY,
    student_id   INT REFERENCES students(student_id),
    issue        TEXT,
    status       VARCHAR(20) DEFAULT 'Pending');
```

5. CONSTRAINTS & TRIGGER

5.1 CHECK Constraint on Menu Items

A CHECK constraint has been added to the menu_items table to enforce a business rule: the price of each food item must always be greater than zero. This ensures valid pricing and prevents incorrect or negative values from being stored in the database.

```
ALTER TABLE menu_items
ADD CONSTRAINT check_price
CHECK (price > 0);
```

5.2 Trigger Function — calc_subtotal()

This PL/pgSQL function automatically calculates the subtotal for each order item before insertion or update. It multiplies the quantity and price of the item, ensuring that subtotal values are always accurate without manual computation.

```
CREATE OR REPLACE FUNCTION calc_subtotal()
RETURNS TRIGGER AS $$
BEGIN
    NEW.subtotal := NEW.quantity * NEW.price;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

5.3 Trigger — trigger_calc_subtotal

The trigger trigger_calc_subtotal fires BEFORE INSERT or UPDATE on the order_items table. It executes the calc_subtotal() function for each row, ensuring that subtotal values are automatically computed and stored.

```
CREATE TRIGGER trigger_calc_subtotal
BEFORE INSERT OR UPDATE ON order_items
FOR EACH ROW
EXECUTE FUNCTION calc_subtotal();
```

5.4 Trigger Function — update_order_total()

This function updates the total_amount field in the orders table whenever order_items are inserted, updated, or deleted. It calculates the sum of all subtotals for a given order, ensuring data consistency.

```
CREATE OR REPLACE FUNCTION update_order_total()
RETURNS TRIGGER AS $$
BEGIN
   UPDATE orders
   SET total_amount = (
      SELECT COALESCE(SUM(subtotal), 0)
      FROM order_items
      WHERE order_id = NEW.order_id
   )
   WHERE order_id = NEW.order_id;

   RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

5.5 Trigger — trigger_update_total

The trigger trigger_update_total fires AFTER INSERT, UPDATE, or DELETE on the order_items table. It ensures that the total order amount is always updated automatically after any change.

```
CREATE TRIGGER trigger_update_total
AFTER INSERT OR UPDATE OR DELETE ON order_items
FOR EACH ROW
EXECUTE FUNCTION update_order_total();
```

5.6 Function — get_user_orders(uid INT)

This stored function accepts a user_id as a parameter and returns all orders placed by that user along with relevant details such as date, status, and total amount.

```
CREATE OR REPLACE FUNCTION get_user_orders(uid INT)
RETURNS TABLE(order_id INT, order_date TIMESTAMP, status VARCHAR,
total_amount DECIMAL) AS $$
BEGIN
   RETURN QUERY
   SELECT order_id, order_date, status, total_amount
   FROM orders
   WHERE user_id = uid;
END;
$$ LANGUAGE plpgsql;
```

6. DATA INSERTION (INSERT QUERIES)

6.1 Inserting User Records

Sample user records are inserted into the users table including basic details such as name, email, phone, and address.

```
INSERT INTO users (name, email, phone, address) VALUES
('Aman', 'aman@gmail.com', '9991110001', 'Delhi'),
('Riya', 'riya@gmail.com', '9991110002', 'Mumbai'),
('Karan', 'karan@gmail.com', '9991110003', 'Bangalore');
```

Showing rows: 1 to 3

	user_id [PK] integer	name character varying (100)	email character varying (100)	phone character varying (15)	address text	created_at timestamp without time zone
1	1	Aman	aman@gmail.com	9991110001	Delhi	2026-04-22 15:46:32.937865
2	2	Riya	riya@gmail.com	9991110002	Mumbai	2026-04-22 15:46:32.937865
3	3	Karan	karan@gmail.com	9991110003	Bangalore	2026-04-22 15:46:32.937865

Output: Users Inserted Successfully

6.2 Inserting Restaurant Records

```
INSERT INTO restaurants (name, location, rating) VALUES
('Dominos', 'Delhi', 4.2),
('KFC', 'Mumbai', 4.0),
('Burger King', 'Bangalore', 4.3);
```

	restaurant_id [PK] integer	name character varying (100)	location character varying (150)	rating numeric (2,1)	is_active boolean	created_at timestamp without time zone
1	1	Dominos	Delhi	4.2	true	2026-04-22 15:46:38.295834
2	2	KFC	Mumbai	4.0	true	2026-04-22 15:46:38.295834
3	3	Burger King	Bangalore	4.3	true	2026-04-22 15:46:38.295834

Output: Restaurants Inserted Successfully

6.3 Inserting Menu Items

```
INSERT INTO menu_items (restaurant_id, item_name, price) VALUES
(1, 'Pizza', 299),
(1, 'Garlic Bread', 149),
(2, 'Fried Chicken', 199),
(3, 'Burger', 129);
```

	item_id [PK] integer	restaurant_id integer	item_name character varying (100)	price numeric (6,2)	is_available boolean
1	1	1	Pizza	299.00	true
2	2	1	Garlic Bread	149.00	true
3	3	2	Fried Chicken	199.00	true
4	4	3	Burger	129.00	true

Output: Menu Items Inserted

6.4 Inserting Orders

```
INSERT INTO orders (user_id, status, total_amount) VALUES
(1, 'Pending', 0),
(2, 'Confirmed', 0);
```

	order_id [PK] integer	user_id integer	order_date timestamp without time zone	status character varying (20)	total_amount numeric (8,2)
1	2	2	2026-04-22 15:46:50.819664	Confirmed	398.00
2	1	1	2026-04-22 15:46:50.819664	Delivered	1046.00

Output: orders Inserted Successfully

6.5 Inserting Order Items

```
INSERT INTO order_items (order_id, item_id, quantity, price) VALUES
(1, 1, 2, 299),
(1, 2, 1, 149),
(2, 3, 2, 199);
```

	order_item_id [PK] integer	order_id integer	item_id integer	quantity integer	price numeric (6,2)	subtotal numeric (8,2)
1	1	1	1	2	299.00	598.00
2	2	1	2	1	149.00	149.00
3	3	2	3	2	199.00	398.00
4	4	1	1	1	299.00	299.00

6.6 Payments

INSERT INTO payments (order_id, payment_method, payment_status, amount) VALUES
(1, 'UPI', 'Completed', 747),
(2, 'Card', 'Pending', 398);

	payment_id [PK] integer	order_id integer	payment_method character varying (20)	payment_status character varying (20)	amount numeric (8,2)	payment_date timestamp without time zone
1	1	1	UPI	Completed	747.00	2026-04-22 15:46:58.330699
2	2	2	Card	Pending	398.00	2026-04-22 15:46:58.330699

6.7 Delivery

INSERT INTO delivery_tracking (order_id, delivery_status) VALUES
(1, 'Preparing'),
(2, 'Out for Delivery');

	delivery_id [PK] integer	order_id integer	delivery_status character varying (20)	delivery_time timestamp without time zone
1	1	1	Preparing	[null]
2	2	2	Out for Delivery	[null]

6.8 Reviews

INSERT INTO reviews (user_id, restaurant_id, rating, review_text) VALUES
(1, 1, 5, 'Great food'),
(2, 2, 4, 'Nice taste'),
(3, 3, 5, 'Excellent');

	review_id [PK] integer	user_id integer	restaurant_id integer	rating integer	review_text text	created_at timestamp without time zone
1	1	1	1	5	Great food	2026-04-22 15:47:07.629301
2	2	2	2	4	Nice taste	2026-04-22 15:47:07.629301
3	3	3	3	5	Excellent	2026-04-22 15:47:07.629301

7. SELECT QUERIES WITH OUTPUTS

Query 1: Orders Placed by User 1

This query retrieves all orders placed by a specific user (user_id = 1). It helps in tracking the order history of a customer.

```
SELECT *
FROM orders
WHERE user_id = 1;
```

	order_id [PK] integer	user_id integer	order_date timestamp without time zone	status character varying (20)	total_amount numeric (8,2)
1	1	1	2026-04-22 15:46:50.819664	Delivered	1046.00

Output: Q1 — Orders of User 1

Query 2: Items in a Specific Order

This query retrieves all food items included in a particular order (order_id = 1) by joining the order_items and menu_items tables.

```
SELECT
   oi.order_item_id,
   m.item_name,
   oi.quantity,
   oi.subtotal
FROM order_items oi
JOIN menu_items m ON oi.item_id = m.item_id
WHERE oi.order_id = 1;
```

Output: Q2 — Items in Order 1

	order_item_id integer	item_name character varying (100)	quantity integer	subtotal numeric (8,2)
1	4	Pizza	1	299.00
2	1	Pizza	2	598.00
3	2	Garlic Bread	1	149.00

Query 3: Restaurant Summary with Ratings

This query retrieves the list of all restaurants along with their average rating and total number of reviews using the restaurant_summary view.

```
SELECT
   restaurant_id,
   name,
   ROUND(avg_rating, 2) AS avg_rating,
   total_reviews
FROM restaurant_summary;
```

Output: Q3 — Restaurant Summary

	restaurant_id integer	name character varying (100)	avg_rating numeric	total_reviews bigint
1	3	Burger King	5.00	1
2	2	KFC	4.00	1
3	1	Dominos	5.00	1

Query 4: Top 3 Restaurants by Rating

This query retrieves the top 3 restaurants based on their average ratings in descending order.

```
SELECT name, round(avg_rating,2)as avg_rating
FROM restaurant_summary
ORDER BY avg_rating DESC
LIMIT 3;
```

Output: Q4 — Top 3 Restaurants

	name character varying (100)	avg_rating numeric
1	Burger King	5.00
2	Dominos	5.00
3	KFC	4.00

Query 5: Total Revenue Generated

This query calculates the total revenue generated from all orders by summing up the total_amount field..

```
SELECT SUM(total_amount) AS total_revenue
FROM orders;
```

Output: Q5 — Total Revenue

	total_revenue numeric
1	1444.00

8. UPDATE QUERIES WITH OUTPUTS

Update Query 1: Mark Order as Delivered

This query updates the status of a specific order (order_id = 1) to 'Delivered'. It is used when an order has been successfully completed.

```
UPDATE orders
SET status = 'Delivered'
WHERE order_id = 1;
```
Output: Complaint #1 status updated to 'Resolved'

```
UPDATE 1
```

SELECT * FROM orders WHERE order_id = 1;

	order_id [PK] integer	user_id integer	order_date timestamp without time zone	status character varying (20)	total_amount numeric (8,2)
1	1	1	2026-04-22 15:46:50.819664	Delivered	1046.00

Output: Order status updated to Delivered

Update Query 2: Mark Pending Orders as Confirmed

This query updates all orders that are currently in 'Pending' status to 'Confirmed'. It is useful for bulk processing of newly placed orders.

```
UPDATE orders
SET status = 'Confirmed'
WHERE status = 'Pending';

SELECT * FROM orders;
```

Output: Pending orders updated to Confirmed

	order_id [PK] integer	user_id integer	order_date timestamp without time zone	status character varying (20)	total_amount numeric (8,2)
1	2	2	2026-04-22 15:46:50.819664	Confirmed	398.00
2	1	1	2026-04-22 15:46:50.819664	Delivered	1046.00

9. DELETE & OTHER QUERIES WITH OUTPUTS

Delete Query: Remove Cancelled Orders

This query deletes all orders that have been marked as 'Cancelled'. It helps in maintaining clean and relevant data in the system.

```
DELETE FROM orders
WHERE status = 'Cancelled';
```
Output: Complaint #1 deleted successfully (1 row affected)

```
DELETE 0
```

Output: Cancelled orders deleted

Advanced Query: Order-wise Item List

This query combines multiple tables using JOIN and uses STRING_AGG to display all items in each order as a single comma-separated list.

```
SELECT
    o.order_id,
    STRING_AGG(m.item_name, ', ') AS items
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN menu_items m ON oi.item_id = m.item_id
GROUP BY o.order_id
ORDER BY o.order_id;
```

Output: Order-wise Item List

	order_id [PK] integer	items text
1	1	Pizza, Garlic Bread, Piz...
2	2	Fried Chicken

10. CONCLUSION

The Food Ordering System successfully demonstrates the application of relational database concepts in a real-world environment. Through this project, we designed and implemented a complete PostgreSQL database that efficiently manages user data, restaurant details, menu items, orders, payments, delivery tracking, and reviews.

The use of foreign key constraints ensured referential integrity across all tables, eliminating the risk of invalid or inconsistent data entries. The implementation of CHECK constraints enforced business rules such as valid pricing, rating limits, and order status control, ensuring data accuracy at the database level.

The triggers implemented for automatic subtotal calculation and order total updates proved to be highly effective in maintaining data consistency and reducing manual computation. The PL/pgSQL stored function get_user_orders() demonstrated how reusable logic can be encapsulated within the database, while the restaurant_summary view provided a simplified way to analyze restaurant performance in real time.

The wide range of SQL queries — from simple SELECT statements to complex JOIN operations, GROUP BY aggregations, and STRING_AGG-based consolidation — highlights the power and flexibility of SQL in handling real-world scenarios efficiently.

Key outcomes of this project include successful management of users, restaurants, and orders, automated calculation of order values using triggers, tracking of payments and delivery status, and implementation of review-based rating systems. Multiple SQL queries were executed to demonstrate all CRUD operations along with advanced querying techniques.

This project provides a strong foundation for developing a full-stack food ordering application that can be extended with modern frontend frameworks, REST APIs, secure authentication systems, and real-time delivery tracking features.

References

- PostgreSQL Documentation — Official PostgreSQL 14 Documentation. https://www.postgresql.org/docs/
- pgAdmin Documentation — https://www.pgadmin.org/docs/
- W3Schools SQL Tutorial — https://www.w3schools.com/sql/
- GeeksforGeeks — PostgreSQL Triggers and Functions — https://www.geeksforgeeks.org/postgresql-trigger/
- TutorialsPoint — SQL Constraints — https://www.tutorialspoint.com/sql/sql-constraints.htm